Geavanceerde Computerarchitectuur Labo 1

Arno De Lathauwer
Indra Coppens

February 2025

1 Context

In dit labo maken we voor het eerst kennis met CUDA. De opdracht is om een programma te schrijven dat wordt uitgevoerd op de GPU en een array omkeert. Als de inputarray bijvoorbeeld {`0, 1, 2, 3`} is, moet de outputarray {`3, 2, 1, 0`} zijn.

2 Methodologie

Als eerste maken we een programma in `C++` dat een array kan omkeren. Dit programma wordt echter uitgevoerd op de CPU en moet dus nog aangepast worden naar een programma dat uitgevoerd kan worden op de GPU. In het aangepaste programma wordt eerst een array van 10 elementen gedefinieerd op de CPU en gekopieerd naar het GPU-geheugen. Vervolgens wordt een CUDA-kernel (`reverseArray`) aangeroepen. Deze kernel laat elke thread twee indices berekenen (`idx` en `opposite`). De `idx` index wordt berekend door eerst de index van het blok te vermenigvuldigen met het aantal threads per blok. Vervolgens wordt de offset van de thread binnen dat blok opgeteld bij het resultaat. Op deze manier krijgt men de globale index van de huidige thread (= `idx`). De opposite index is de tegenoverstaande index van idx (rond het middelste element in de array als "symmetrie-as"). Op die manier wordt het eerste element door de eerste thread gewisseld met het laatste element, het tweede element wordt gewisseld met het voorlaatste element enzovoort. De twee elementen horend bij deze indices worden dus verwisselt om zo de array om te keren. De kernel wordt uitgevoerd met één blok van vijf threads. Elke thread is dus verantwoordelijk voor het omwisselen van twee elementen. Er kunnen ook meerdere blokken gebruikt worden, aangezien we bij het berekenen van de indices rekening houden met de index van het blok. Na uitvoering wordt de gewijzigde array terug gekopieerd naar de CPU en uitgeprint. Tot slot wordt het GPU-geheugen vrijgemaakt. De code kan gevonden worden in Appendix A.

3 Observaties/resultaten

In de code wisselt elke thread twee integers van plaats. Wanneer we werken met 10 integers, hebben we dus minimum 5 threads nodig om de volledige array om te keren. Indien we minder dan 5 threads gebruiken zien we dat slechts een deel van de elementen in de array wordt verwisseld, maar dat het midden van de array onveranderd blijft.

De huidige GPUs hebben een maximum van 1024 threads per block [1]. Indien we een groter aantal threads proberen kiezen, zal het programma geen errors geven, maar wordt de functie niet uitgevoerd. Het resultaat is dat de array in zijn oorspronkelijke volgorde wordt weergegeven. Het totaal aantal threads kan wel hoger zijn dan 1024 indien er meerdere blokken worden gebruikt.

4 Conclusie

In dit labo hebben we geleerd hoe we CUDA kunnen gebruiken om parallelle berekeningen op de GPU uit te voeren. We hebben een programma geschreven dat een array omkeert door gebruik te maken van threads op de GPU. Elke thread wisselt twee elementen van plaats. Het voordeel van elementen te verwisselen, gebruik makend van meerdere threads, is dat de array in parallel kan worden omgedraaid. Dit is veel sneller dan het omdraaien van de array op een sequentiële manier. Daarnaast hebben we geobserveerd dat het aantal threads per block gelimieteerd is op 1024 threads per block.

Appendix A: Code omdraaien array

Deze CUDA-code keert een array om door gebruik te maken van parallelle verwerking op de GPU. De code is gebaseerd op de voorbeelden uit [1] en [2].

```

#include <iostream>
#include <cuda_runtime.h>

#define N 10

__global__ void reverseArray(int *d_arr, int size) {
    // BlockDim.x is de grote van een block, aantal threads per block
    // ThreadIdx.x is index binnen een  block
    // BlockIdx.x is index van een block
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int opposite = size - 1 - idx;

    if (idx < size / 2) {
        int temp = d_arr[idx];
        d_arr[idx] = d_arr[opposite];
        d_arr[opposite] = temp;
    }
}

int main() {
    int h_arr[N] = {1,2,3,4,5,6,7,8,9,10}; // Host-array (CPU)

    int *d_arr; // Device-array (GPU)
    cudaMalloc((void**)&d_arr, N * sizeof(int));
    cudaMemcpy(d_arr, h_arr, N * sizeof(int), cudaMemcpyHostToDevice);

    // Kernel aanroepen <<<blocks per grid, threads per block>>>
    reverseArray<<<1, 5>>>(d_arr, N);
    cudaDeviceSynchronize();

    // Gekopieerde array terug naar de host
    cudaMemcpy(h_arr, d_arr, N * sizeof(int), cudaMemcpyDeviceToHost);

    % Printen van de output array
    for (int i = 0; i < N; i++) {
        std::cout << h_arr[i] << " ";
    }

    // Opruimen
    cudaFree(d_arr);

    return 0;
}
```

References

[1] 'CUDA C++ Programming Guide'. Accessed: Feb. 11, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[2] Pceeckishan, 'Fundamental Array Operations on GPUs with CUDA', Medium. Accessed: Feb. 11, 2025. [Online]. Available: https://medium.com/@2019pceeckishan31/introduction-to-cuda-basic-array-operations-on-gpu-6cdbe0684842